MASTÈRE EXPERT EN DÉVELOPPEMENT ET DÉPLOIEMENT DE SOLUTIONS NUMÉRIQUES SÉCURISÉES · RNCP 40166







CI/CD

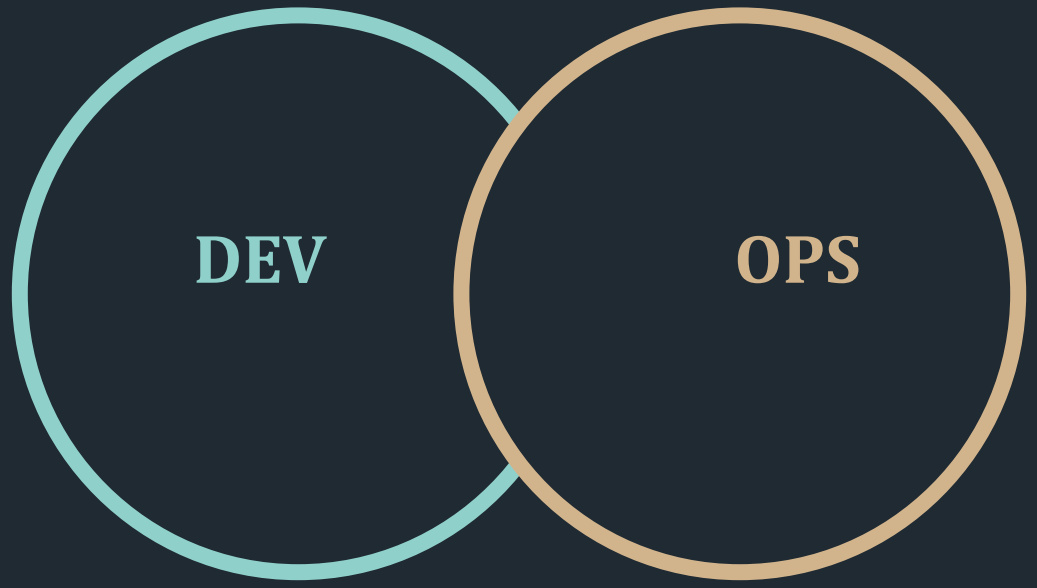
Industrialiser, automatiser et sécuriser le cycle de vie du logiciel

JOUR 1 · Fondamentaux, qualité, conteneurs et pipeline Jenkins

Formateur : Alimou Diallo · 09/11/2026 · Bloc 4 – E46 · Sup de Vinci Rennes



DevOps : une culture avant d'être des outils



C — **Culture**
collaboration, responsabilité partagée

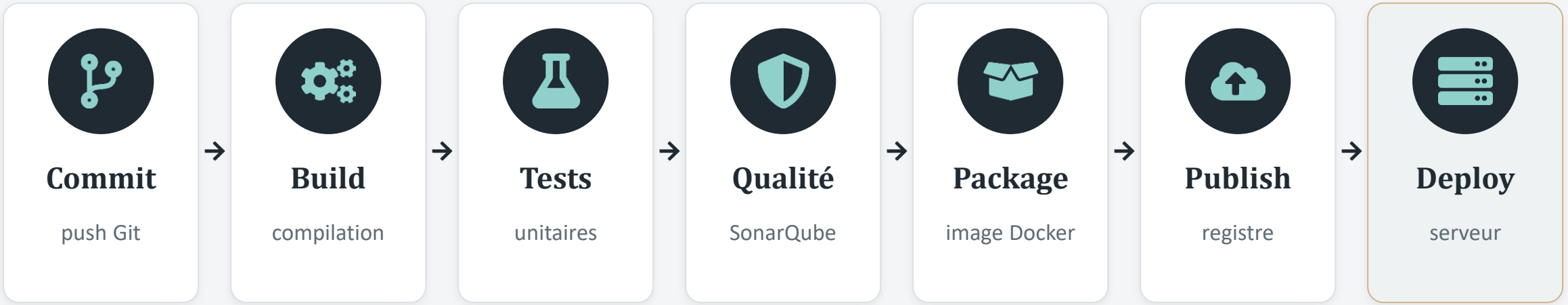
A — **Automatisation**
build, tests, déploiement

L — **Lean**
petits lots, flux continu

M — **Mesure**
métriques, feedback rapide

S — **Sharing**
connaissances, documentation

Le pipeline : du commit à la production



Jour 1

Commit - Build - Tests - Qualité - Package - Publish

Jour 2

Provisionnement Terraform, configuration Ansible, déploiement sur serveur

CI, livraison continue, déploiement continu



CI

Intégration continue

Chaque commit est fusionné, compilé et testé automatiquement.

Feedback en minutes · build toujours sain



CD

Livraison continue

Artefact toujours prêt à être déployé ; mise en prod déclenchée par une validation manuelle.

Approbation humaine avant la prod



CD

Déploiement continu

Chaque changement validé par la pipeline part en production sans intervention.

Exige tests et monitoring très robustes

Tester à chaque niveau : la pyramide des tests



Non-régression
Suite rejouée à chaque commit

Couverture
Indicateur de confiance, pas un but

Fail fast
Tests rapides en premier

SonarQube : l'analyse statique comme barrière



Bugs
Erreurs de logique

Vulnérabilités
Failles de sécurité

Code smells
Dette technique

Couverture
% de code testé

Duplications
Code copié-collé

Analyse manuelle (sonar-scanner)

```
sonar-scanner \
  -Dsonar.projectKey=mon-backend \
  -Dsonar.sources=src \
  -Dsonar.host.url=http://sonar:9000 \
  -Dsonar.token=$SONAR_TOKEN
```

Quality Gate

- Couverture sur code neuf ≥ 80 %
- Aucune vulnérabilité bloquante
- Duplications < 3 %
- Échec = build rouge, déploiement bloqué
- Politique de validation automatique avant déploiement

Scanners de code et validation automatique

Code (SAST)
SonarQube

Dépendances
dependency-check, npm audit

Secrets
gitleaks

Image Docker
Trivy

```
gitleaks detect --source . --exit-code 1
trivy fs --exit-code 1 --severity HIGH,CRITICAL .
trivy image --exit-code 1 \
  --severity CRITICAL user/backend:1.0.0
```

Le code de sortie non nul fait échouer le stage : le déploiement est bloqué.

Politique de validation

- Branche main protégée
- Quality Gate vert pour fusionner
- Scanners sans alerte critique
- Non-régression réussie

Jenkins : orchestrateur de pipelines



Jenkins

Serveur d'automatisation open source, extensible par plugins.

Controller

Interface, configuration, ordonnancement des jobs

Agents / nœuds

Exécutent les étapes (machine, conteneur Docker)

Plugins

Git, Pipeline, Docker, SonarQube, Credentials Binding

Pipeline as Code

Jenkinsfile versionné avec le code source

Le Jenkinsfile : le pipeline comme code

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh './mvnw -B package' }
    }
    stage('Test') {
      steps { sh './mvnw test' }
    }
    stage('Quality') {
      steps {
        withSonarQubeEnv('sonar') {
          sh './mvnw sonar:sonar'
        }
      }
    }
  }
  post { failure { echo 'Pipeline en échec' } }
}
```

Variante Node.js : npm ci · npm test · npm run build (même structure de pipeline)

`agent`
Où exécuter

`stages / stage`
Les étapes du flux

`steps`
Les commandes

`post`
Actions finales (succès, échec)

`environment`
Variables et credentials

Docker : image, conteneur, Dockerfile



Dockerfile
recette de construction

Image
modèle immuable, en couches

Conteneur
instance isolée en exécution

Dockerfile multi-stage

```
# Étape 1 : build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn -B package -DskipTests

# Étape 2 : exécution légère
FROM eclipse-temurin:21-jre
COPY --from=build /app/target/*.jar app.jar
USER 1001
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

Dockerfile : les bonnes pratiques

Images légères

Base slim ou alpine, multi-stage pour exclure les outils de build

Utilisateur non-root

USER dédié : réduit la surface d'attaque

Tags explicites

Jamais latest en production : versions fixes ou digest

Cache des couches

Copier d'abord les dépendances, puis le code

.dockerignore

Exclure .git, secrets, node_modules, fichiers de test

Scan de l'image

Détection de vulnérabilités avant publication (Trivy)

Le registre : publier et distribuer les images

1

Build

```
docker build -t
user/backend:1.0.0 .
```

2

Login

```
docker login -u user
(token)
```

3

Push

```
docker push
user/backend:1.0.0
```

4

Pull

```
docker pull
user/backend:1.0.0
```

Stratégie de tags

- Version sémantique : 1.4.2
- Identifiant du commit : sha-a1b2c3d
- latest réservé aux tests



Docker Hub

Dépôt public ou privé · Access Token · limites de pulls pour les comptes gratuits

Gestion des secrets dans la pipeline

À proscrire

- Mot de passe en clair dans le Jenkinsfile
- Fichier .env commité dans Git
- Secret en ARG ou ENV du Dockerfile
- Secret affiché dans les logs (echo)
- Compte unique partagé, sans rotation

Bonnes pratiques

- Credentials Jenkins + masquage des logs
- Tokens à portée minimale, avec expiration
- Rotation régulière, coffre (Vault)

```
withCredentials([usernamePassword(
  credentialsId: 'dockerhub',
  usernameVariable: 'DH_USER',
  passwordVariable: 'DH_TOKEN')]) {
  sh 'echo $DH_TOKEN | docker login \
  -u $DH_USER --password-stdin'
}
```

Pipelines réutilisables : composants partagés



Jenkins

Shared Library

Fonctions Groovy partagées

```
@Library('ci-lib') _

pipelineBackend(
  app: 'backend',
  image: 'user/backend'
)
```



GitLab CI

include / templates

Gabarits centralisés

```
include:
  - project: 'org/ci-templates'
    ref: v1
    file: '/backend.yml'
```



GitHub Actions

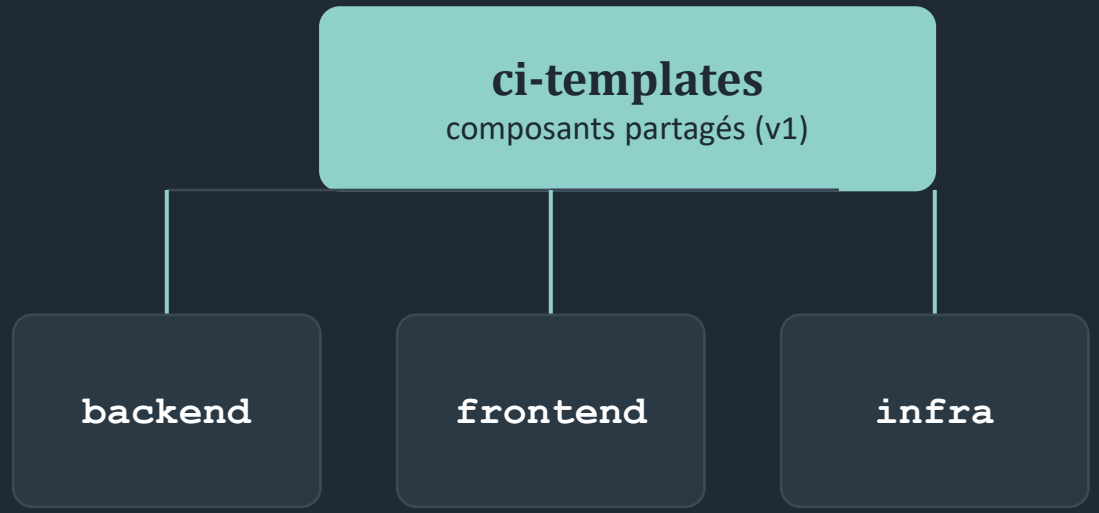
Composite · reusable

Actions et workflows partagés

```
jobs:
  ci:
    uses: org/ci-templates/
      .github/workflows/ci.yml@v1
```

Principe DRY : écrire le pipeline une fois, l'appeler partout avec une version (v1) figée.

Orchestration multi-dépôts (multi-repo)



backend et frontend appellent les composants partagés.

Le pipeline du backend déclenche le dépôt infra pour déployer.

Déclencher un autre dépôt

```
# GitLab CI
deploy-infra:
  stage: deploy
  trigger:
    project: org/infra
    branch: main
```

```
// Jenkins
build job: 'deploy-infra',
  parameters: [string(name: 'TAG',
               value: env.TAG)]
```

FIL ROUGE

Notre architecture cible sur les 2 jours

JOUR 1 · AUJOURD'HUI



Git → 

Jenkins → 

SonarQube → 

Docker Hub

JOUR 2 · DEMAIN



GitLab → 

Terraform → 

Ansible → 

Serveur

DÉMARCHE

Du manuel à l'automatique : notre progression

1 Manuel

Lancer le build, les tests et l'analyse SonarQube à la main, lire les résultats.

COMPRENDRE

→

2 Scripté

Conteneuriser le backend, construire et publier l'image sur Docker Hub.

REPRODUIRE

→

3 Automatisé

Jenkinsfile : build, tests, qualité, image, push, déclenchés à chaque commit.

INDUSTRIALISER

Contexte : backend d'une application Full Stack. Le cahier des charges du TP est remis séparément.

Documenter et préparer la certification

README du pipeline

Prérequis, variables, déclencheurs, schéma

Procédure de déploiement

Étapes reproductibles par un tiers

Runbook

Quoi faire quand une étape échoue

Argumentaire MSPR 4.1

Valeur ajoutée de l'automatisation

Documentation générée automatiquement par le pipeline

```
mvn javadoc:javadoc                                    # Java
npx @redocly/cli build-docs openapi.yaml -o docs/index.html
```